



**UNIVERSIDADE FEDERAL DE
MINAS GERAIS**

TRABALHO FINAL DE AUTOMAÇÃO EM TEMPO REAL

Entrega 1: Definição da Arquitetura

Ana Clara Melado, 2023421386

Bruno Araujo Pinto, 2023038965

Mariana Villefort Rabelo, 2022061033

Belo Horizonte
22 de junho de 2026

1 Introdução

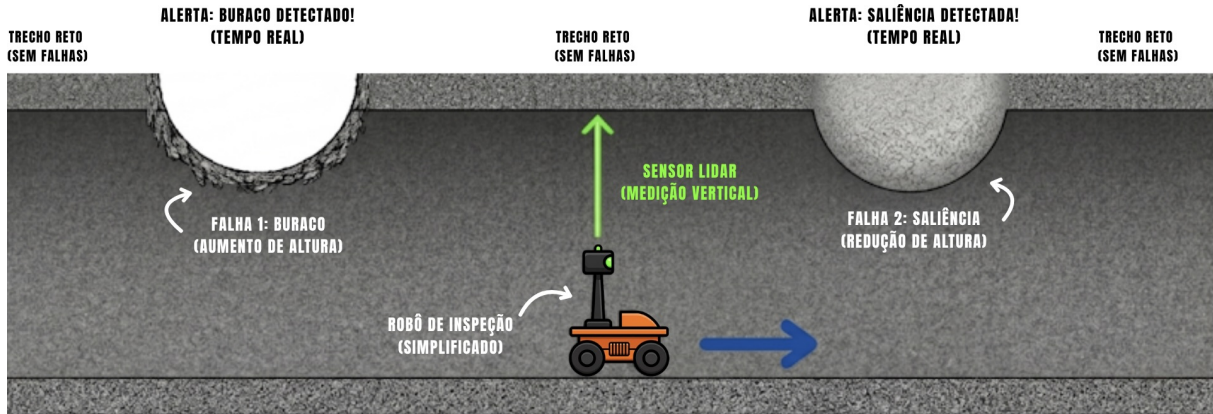


Figura 1: Túnel (vista lateral 2D)

A inspeção periódica de túneis e estruturas civis é fundamental para garantir a segurança operacional e estrutural desses ambientes. Entretanto, métodos tradicionais de inspeção manual apresentam limitações relacionadas ao custo, ao tempo de execução e à exposição de operadores a ambientes potencialmente perigosos. Nesse contexto, sistemas autônomos de inspeção vêm sendo cada vez mais utilizados, permitindo a coleta contínua de dados e a identificação precoce de falhas estruturais.

Este trabalho tem como objetivo o desenvolvimento da arquitetura de um sistema embarcado para inspeção automática de túneis, utilizando conceitos de Automação em Tempo Real, programação concorrente e sincronização entre tarefas. O sistema proposto simula o funcionamento de um robô autônomo responsável por percorrer um túnel realizando medições da superfície do teto por meio de sensores simulados, como encoder e LIDAR, possibilitando a detecção de irregularidades estruturais.

A implementação desenvolvida na primeira etapa do projeto concentra-se principalmente nas tarefas responsáveis pelo controle de navegação, cálculo de distância percorrida, reconstrução da superfície do túnel e coleta de dados. Para isso, foram utilizados processos e múltiplas threads em C++, além de mecanismos de sincronização como `mutex` e `condition_variable`, garantindo acesso seguro aos buffers compartilhados e evitando condições de corrida durante a troca de informações entre produtor e consumidor.

Além da implementação dos buffers compartilhados, o projeto também realiza testes preliminares de sincronização e bloqueio, verificando situações de buffer vazio e buffer cheio. Esses testes permitem validar o comportamento concorrente do sistema e demonstrar a correta coordenação entre as tarefas executadas em paralelo, aspecto essencial em aplicações de tempo real.

Por fim, esta etapa do trabalho estabelece a base arquitetural necessária para as próximas implementações do sistema completo, incluindo comunicação remota, interface gráfica de operação e integração com protocolos de comunicação.

2 Proposta de Arquitetura

A proposta de arquitetura apresentada na primeira etapa do trabalho foi totalmente remodelada, pois não se adequava aos requisitos de utilização da biblioteca Boost. Além

disso, identificou-se o acúmulo de tarefas na fila entre as *threads*. Na abordagem inicial, o travamento das tarefas era frequente, dado que o período da tarefa *distancia_percorrida()* é de 20 ms, enquanto o da tarefa *reconstrucao_teto()* é de 100 ms. Consequentemente, a taxa de escrita no `BUFFER_NAVEGACAO` é significativamente superior à sua taxa de leitura. Assim, ao utilizar a instrução *wait()* em *distancia_percorrida()*, o sistema entrava em um *deadlock*. Para evitar esse cenário, quando uma tarefa agendada não está apta para execução, ela é finalizada forçadamente, sendo contabilizada como uma perda de prazo (*miss*).

Ademais, implementou-se o uso de *strands* para otimizar o aproveitamento da biblioteca Boost e evitar condições de corrida no acesso aos *buffers*. A classe `io_service::strand` atua como um serializador de contexto de I/O, gerindo o agendamento de execuções. Por meio da função `boost::post`, é possível enfileirar novas tarefas de forma segura. Isso garante que as tarefas sejam executadas sequencialmente na *strand*, eliminando a preempção concorrente e as condições de corrida sem a necessidade de bloqueios explícitos.

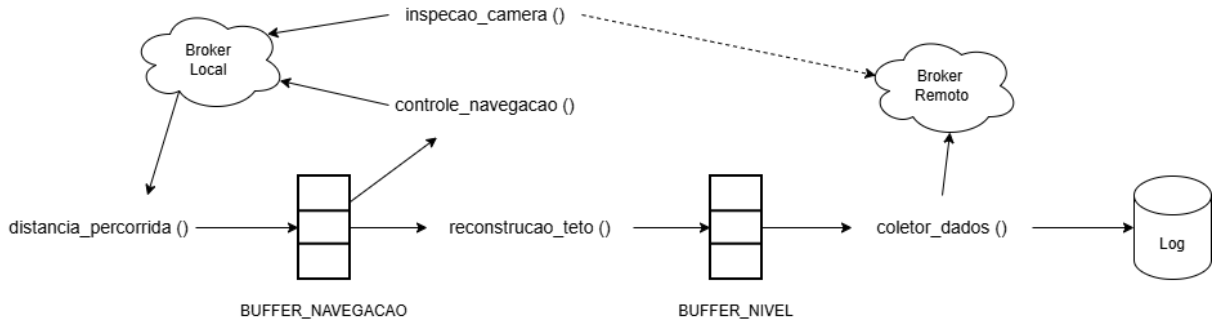


Figura 2: Proposta de arquitetura para o trabalho prático. É possível identificar a relação dos buffers com as funções.

2.1 Periodização

A sincronização das tarefas como introduzido anteriormente foi totalmente readequado para a biblioteca Boost. Foram criadas três strands: *strand_navegacao*, *strand_processamento* e *strand_camera*. As duas primeiras gerenciam as tarefas como ilustrado na figura 3. Já a terceira gerencia apenas a utilização da câmera que não interage diretamente com as outras tarefas.

Ademais, como o período das strands são conflitantes ($T_{distancia_percorrida} = 20ms$ e $T_{reconstrucao_teto} = 100ms$), a tarefa *distancia_percorrida* acontece em único ciclo, enquanto a tarefa *reconstrucao_teto* executa em "lotes", até que se encontre uma falha. Dessa forma, é possível facilitar a sincronização de ambas as filas.

2.2 Comunicação

Como requisitado pelo enunciado do trabalho, é necessário implementar pelo menos uma comunicação interprocesso (IPC) e implementar signal entre duas tarefas, para a utilização da câmera.

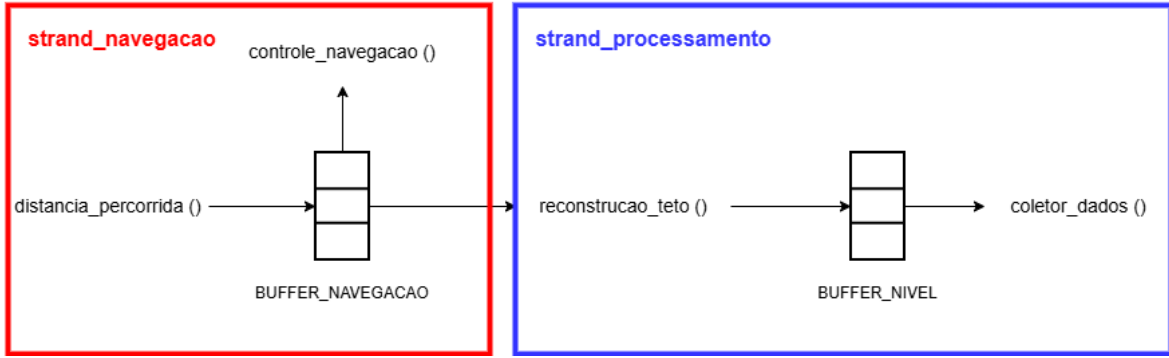


Figura 3: Divisão das strands.

As tarefas *comando_navegacao* e *controle_navegacao* foram implementadas em processos diferentes. A troca de informações entre eles é feita por meio da Memória Compartilhada, implementada na própria main. A Memória Compartilhada foi implementada por meio da Biblioteca Boost, que cria um FileDescriptor para troca de informações. Essa abordagem adaptada foi necessária para haver compatibilidade com o sistema MacOS. Porém, a utilização se mantém da mesma forma. Já a utilização do signal foi utilizado entre para comunicar a presença de uma falha no túnel. Em *reconstrucao_teto*, ao identificar uma variação acima da esperada, o signal `SIGUSR1` é acionado. A função *signal_handler* trata esse signal, desacelerando o carrinho e acionando a câmera.

3 Resultados de Testes Preliminares Realizados

Durante a fase de desenvolvimento da arquitetura do sistema, foram realizados testes preliminares para validar a implementação dos buffers compartilhados e a sincronização entre as tarefas que escrevem e leem nos buffers. Esses testes foram fundamentais para garantir que o sistema se comporta corretamente em diferentes condições do sistema, evitando, por exemplo, condições de corrida, inanição e garantindo a integridade dos dados e a execução e finalização corretas de todas as partes já implementadas do código.

Os testes foram conduzidos com dados simulados, uma vez que o restante do código ainda será realizado na etapa posterior do projeto. Foram feitos testes de execução, de diferentes quantidades de falhas encontradas no caminho percorrido, casos onde o buffer estava vazio, garantindo que as tarefas consumidoras aguardavam corretamente até que os dados estivessem disponíveis, além de casos de bloqueio de leitores e escritores do buffer de navegação, garantindo a boa execução do meta-locking implementado, em que mais de uma thread pode ler o buffer, porém somente uma thread pode acessá-lo em caso de escrita.

Os resultados desses testes preliminares foram positivos, demonstrando que a implementação dos buffers compartilhados e a sincronização entre as tarefas estão funcionando conforme o esperado. Esses resultados fornecem uma base sólida para a próxima etapa do desenvolvimento do sistema, onde será finalizada a implementação do sistema.

3.1 Compilação e Execução Inicial

Inicialmente, foi validado o processo de compilação do programa por meio do arquivo `Makefile`, que foi comprovada pela utilização de logs que imprimem mensagens indicando tarefas que foram realizadas. Durante a execução inicial, verificou-se a criação do processo associado ao comando de navegação, a escrita de comandos em memória compartilhada e a leitura desses comandos pelo processo principal. Foram observados valores coerentes para o comando de operação automática e para o setpoint de velocidade, indicando que a estrutura inicial de compartilhamento de informações foi inicializada corretamente.

Em seguida, foram criadas as threads correspondentes às principais tarefas do sistema: cálculo de distância percorrida, inspeção por câmera, coletor de dados, reconstrução do teto e controle de navegação. A criação das threads foi acompanhada por mensagens de log, permitindo verificar a inicialização e execução das tarefas.

Após isso, foram impressos diversos logs da execução, que serão melhor explorados nos próximos testes.

3.2 Teste sem Detecção de Falhas

O primeiro teste funcional teve como objetivo verificar o comportamento do sistema sem que houvesse detecção de falhas na superfície do teto. Nesse caso, a tarefa de reconstrução do teto produziu números simulados de distância (que futuramente virão do LIDAR) para o `BUFFER_NIVEL`, enquanto o coletor de dados realizou a leitura dessas informações.

Durante a execução, observou-se que não houve acionamento indevido da câmera nem redução de velocidade associada à inspeção, o que indica que a lógica de falha permaneceu inativa durante o teste. Ao final da execução, todas as threads foram finalizadas corretamente e os buffers finais foram impressos, demonstrando que a ausência de falhas não provocou bloqueio permanente no sistema.

3.3 Teste com uma Falha Detectada

Esse teste teve como objetivo validar o comportamento do sistema diante da detecção de uma única falha na superfície do teto. Para isso, a lógica de detecção da tarefa de reconstrução foi configurada para gerar um evento de falha em apenas uma iteração durante toda a execução.

Durante o teste foi registrada a detecção de falha em um único instante da execução, seguida do acionamento lógico da câmera e da redução do setpoint de velocidade. Após o tratamento do evento, a execução prosseguiu normalmente.

Ao final, todas as threads foram encerradas sem travamentos, indicando que o sistema foi capaz de tratar uma falha isolada sem comprometer a execução das demais tarefas.

Além disso, foi possível verificar a tarefa de inspeção por câmera a partir da geração de eventos de falha pela tarefa de reconstrução do teto. Verificou-se que a câmera não executava atividades enquanto não havia falhas detectadas. Quando uma falha foi simulada, a reconstrução incrementou o contador de eventos de câmera, atualizou os estados compartilhados e notificou a variável de condição associada à inspeção.

Durante os testes, foram registradas mensagens indicando o início e a finalização da inspeção por câmera. Esse comportamento confirmou que a thread da câmera foi corretamente desbloqueada diante de eventos de falha e que retornou ao estado inativo após o processamento.

3.4 Teste com Múltiplas Falhas

Outro teste realizado foi o de múltiplas falhas, no qual a tarefa de reconstrução do teto foi configurada para simular a detecção de falha em todas as iterações. O objetivo desse teste foi verificar se o sistema seria capaz de lidar com acionamentos sucessivos da lógica de inspeção.

Durante a execução, foram registradas múltiplas mensagens de falha detectada, cada uma associada ao acionamento lógico da câmera e à redução do setpoint de velocidade. Mesmo com sucessivos eventos de inspeção, o sistema continuou realizando a escrita e leitura dos dados no buffer de nível sem apresentar erros.

Esse teste permitiu validar o funcionamento da câmera sem perdas de eventos quando múltiplas falhas são detectadas em sequência. Ao final da execução, a thread de reconstrução foi finalizada, a thread de câmera encerrou corretamente e todas as demais tarefas concluíram sua execução. Dessa forma, o teste indicou que o sistema é capaz de tratar múltiplos eventos de falha sem bloqueio permanente.

3.5 Teste de Buffer de Nível Vazio

O teste de buffer de nível vazio teve como objetivo verificar se uma tarefa consumidora permanece bloqueada quando não há dados disponíveis para leitura. Esse comportamento foi observado na tarefa coletora de dados, responsável por consumir informações do `BUFFER_NIVEL`.

Durante a execução, o coletor registrou mensagens indicando que o buffer estava vazio e permaneceu aguardando a chegada de novos dados. Após a tarefa de reconstrução do teto escrever um novo dado e notificar a variável de condição correspondente, o coletor retomou a execução e realizou a leitura do dado disponível.

Esse comportamento demonstra que o consumidor não realiza leituras inválidas quando o buffer está vazio. Em vez disso, a thread permanece bloqueada, sem espera ativa, até que o produtor disponibilize uma nova amostra. Assim, o teste validou o tratamento correto da condição de buffer vazio.

3.6 Teste de Buffer de Nível Cheio

O teste de buffer de nível cheio foi realizado com o objetivo de verificar se a tarefa produtora é corretamente bloqueada quando o buffer atinge sua capacidade máxima. Para isso, a tarefa de reconstrução do teto foi configurada para produzir dados em uma taxa superior à taxa de consumo do coletor de dados.

Durante a execução, o `BUFFER_NIVEL` foi sendo ocupado a cada iteração até atingir sua capacidade máxima. Nesse momento, a tarefa de reconstrução registrou a condição de buffer cheio e permaneceu bloqueada aguardando a liberação de espaço.

Quando o coletor de dados consumiu as amostras (como a execução é muito rápida, na figura todas foram consumidas antes da sinalização), a tarefa de reconstrução foi notificada e retomou a escrita no buffer. Esse comportamento confirma que o produtor não sobrescreve dados quando o buffer está cheio, permanecendo em espera até que o consumidor libere espaço. Assim, a lógica produtor-consumidor implementada para o buffer de nível foi validada.

3.7 Teste de Bloqueio de Leitores e Escritores

Foi realizado também um teste específico para o `BUFFER_NAVEGACAO`, cujo acesso é controlado por uma lógica de leitores e escritores usando meta-locking. Esse buffer é escrito pela tarefa de cálculo de distância percorrida e lido pelas tarefas de controle de navegação e reconstrução do teto.

Para validar o mecanismo de sincronização, foram introduzidos atrasos artificiais nas regiões críticas de leitura e escrita. Inicialmente, a tarefa de distância percorrida manteve o lock de escrita com o escritor por um intervalo maior, fazendo com que as tarefas leitoras aguardassem a liberação do recurso. Os logs indicaram que tanto a reconstrução do teto quanto o controle de navegação permaneceram bloqueados enquanto havia um escritor ativo.

Em seguida, observou-se também a situação inversa: a tarefa escritora aguardou a liberação do buffer enquanto existiam leitores ativos que foram colocados para dormir propositalmente, levando ao atraso da liberação do lock. Após a finalização das leituras, o escritor retomou a execução e realizou a escrita no buffer.

Esse teste demonstrou que a lógica de meta-locking implementada no `BUFFER_NAVEGACAO` impede leituras durante uma escrita ativa e impede escritas enquanto há leitores ativos. Dessa forma, foi validada a exclusão adequada entre leitores e escritores, reduzindo o risco de inconsistências no acesso ao buffer compartilhado.

3.8 Teste de Finalização das Threads

Outro aspecto avaliado foi a finalização controlada das threads. Durante os testes iniciais, observou-se que algumas threads poderiam permanecer bloqueadas indefinidamente caso ficassem aguardando eventos que não seriam mais produzidos, causando deadlock. Esse comportamento foi corrigido por meio da inclusão de flags de finalização e notificações adicionais ao término das tarefas produtoras.

Nos testes finais, a função principal conseguiu aguardar a finalização de todas as threads por meio de `join()`. Foram registradas mensagens indicando a conclusão das threads de distância percorrida, inspeção por câmera, coletor de dados, reconstrução do teto e controle de navegação, seguida da impressão dos valores escritos em cada buffer e da finalização do sistema.

A thread de controle de navegação permaneceu ativa por mais tempo devido a atrasos artificiais inseridos para simular um consumidor mais lento, mas finalizou corretamente ao término de suas iterações. Dessa forma, foi validado que o sistema encerra de maneira controlada, sem a necessidade de interrupção manual, já que todas as tarefas possuem uma condição de término definida.

3.9 Considerações Finais dos Testes

De forma geral, os testes preliminares indicaram que a implementação atual atende aos objetivos definidos para a primeira etapa do projeto. Foram validadas a criação das tarefas concorrentes, a comunicação por buffers, a sincronização por mutexes e variáveis de condição, o bloqueio correto em situações de buffer cheio e vazio, o controle de leitores e escritores no buffer de navegação, o acionamento da câmera por evento simulado e a finalização controlada das threads.

Apesar disso, os testes ainda foram realizados com dados simulados e em ambiente controlado. A validação completa do sistema dependerá da etapa seguinte.

Todos os prints e testes realizados encontram-se no apêndice desse relatório.

4 Conclusão

Na primeira etapa do trabalho foi desenvolvida a arquitetura inicial do sistema de inspeção automática de túneis, contemplando a implementação das principais tarefas concorrentes responsáveis pela navegação, reconstrução do teto, coleta de dados e inspeção por câmera. O sistema foi estruturado utilizando processos, múltiplas threads e mecanismos de comunicação e sincronização em C++, com foco na aplicação dos conceitos estudados na disciplina de Automação em Tempo Real.

A comunicação entre processos foi realizada por meio de memória compartilhada, enquanto a troca de dados entre tarefas concorrentes foi organizada utilizando buffers compartilhados protegidos por `mutexes` e variáveis de condição. Para o `BUFFER_NAVEGACAO`, foi implementada uma lógica de leitores e escritores utilizando meta-locking, permitindo múltiplas leituras simultâneas e garantindo exclusão mútua durante operações de escrita. Já o `BUFFER_NIVEL` foi estruturado no modelo produtor-consumidor, controlando corretamente situações de buffer cheio e buffer vazio.

Além da implementação da arquitetura básica do sistema, foram realizados diversos testes preliminares para validar o comportamento concorrente da aplicação. Os testes demonstraram que as tarefas conseguem trocar informações corretamente, que os mecanismos de sincronização evitam condições de corrida e que os eventos de acionamento da câmera são tratados adequadamente. Também foi validada a finalização controlada das threads e o correto funcionamento dos buffers compartilhados em diferentes cenários de execução.

Os resultados obtidos até o momento indicam que a base estrutural do sistema está funcionando de forma coerente e atende aos objetivos definidos para a primeira etapa do projeto. Apesar disso, parte das funcionalidades ainda utiliza dados simulados e simplificações temporárias, principalmente relacionadas à física da navegação, ao processamento da câmera e à interface de operação remota.

Como próximos passos, pretende-se concluir a integração completa da simulação física do robô e do túnel, implementar a comunicação remota via protocolo MQTT e desenvolver as interfaces gráficas de simulação e operação remota. Além disso, serão incorporados testes mais avançados de desempenho, tempo de resposta e escalonabilidade das tarefas, avaliando o comportamento do sistema sob diferentes cargas de processamento e frequências de execução. Também serão realizados testes adicionais de robustez e sincronização para validar o funcionamento do sistema completo em cenários mais próximos de uma aplicação real.

5 Apêndice

5.1 Testes realizados

```
● aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[19:12:47] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
```

Figura 4: Execução inicial do programa

```
[19:12:47] [MAIN] Buffers inicializados
[19:12:47] [MAIN] Criando thread distancia_percorrida
[19:12:47] [MAIN] Criando thread inspecao_camera
[19:12:47] [MAIN] Criando thread coletor_dados
[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada
```

Figura 5: Criação das Threads

```
TESTE SEM DETECÇÃO DE FALHAS

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[17:51:45] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[17:51:45] [MAIN] Buffers inicializados
[17:51:45] [MAIN] Criando thread distancia_percorrida
[17:51:45] [MAIN] Criando thread inspecao_camera
[17:51:45] [MAIN] Criando thread coletor_dados
[17:51:45] [MAIN] Criando thread reconstrucao_teto
[17:51:45] [MAIN] Criando thread controle_navegacao
[17:51:45] [MAIN] Esperando thread 0
[17:51:45] [COLETOR] Thread inicializada
[17:51:45] [RECONSTRUCAO] Thread inicializada
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [CONTROLE] Posição lida (navegação): 0.000000
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 10.782785
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 11.484781
```

[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 26.471970
 [17:51:45] [COLETOR] Posição lida (nível): 10.782785
 [17:51:45] [COLETOR] Posição lida (nível): 11.484781
 [17:51:45] [COLETOR] Posição lida (nível): 26.471970
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 39.097961
 [17:51:45] [COLETOR] Posição lida (nível): 39.097961
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 68.813644
 [17:51:45] [COLETOR] Posição lida (nível): 68.813644
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 64.550240
 [17:51:45] [COLETOR] Posição lida (nível): 64.550240
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 58.606701
 [17:51:45] [COLETOR] Posição lida (nível): 58.606701
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 59.807774
 [17:51:45] [COLETOR] Posição lida (nível): 59.807774
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 63.561176
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 41.966251
 [17:51:45] [COLETOR] Posição lida (nível): 63.561176
 [17:51:45] [COLETOR] Posição lida (nível): 41.966251
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.831238
 [17:51:45] [COLETOR] Posição lida (nível): 53.831238
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 69.043571
 [17:51:45] [COLETOR] Posição lida (nível): 69.043571
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 32.942486
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 31.726030
 [17:51:45] [COLETOR] Posição lida (nível): 32.942486
 [17:51:45] [COLETOR] Posição lida (nível): 31.726030
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 96.737633
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 42.435276
 [17:51:45] [COLETOR] Posição lida (nível): 96.737633
 [17:51:45] [COLETOR] Posição lida (nível): 42.435276
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.318428
 [17:51:45] [COLETOR] Posição lida (nível): 53.318428
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 43.819653
 [17:51:45] [COLETOR] Posição lida (nível): 43.819653
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 86.956482
 [17:51:45] [COLETOR] Posição lida (nível): 86.956482
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 38.257179

```

[17:51:45] [RECONSTRUCAO] Thread finalizada
[17:51:45] [COLETOR] Posição lida (nível): 38.257179
[17:51:46] [DISTANCIA] Posição escrita (navegação): 28.958828
[17:51:47] [CONTROLE] Posição lida (navegação): 0.000000
[17:51:47] [DISTANCIA] Posição escrita (navegação): 93.278160
[17:51:48] [DISTANCIA] Posição escrita (navegação): 5.549335
[17:51:49] [CONTROLE] Posição lida (navegação): 5.549335
[17:51:49] [DISTANCIA] Posição escrita (navegação): 59.613941
[17:51:50] [DISTANCIA] Posição escrita (navegação): 5.053784
[17:51:51] [CONTROLE] Posição lida (navegação): 59.613941
[17:51:51] [DISTANCIA] Posição escrita (navegação): 96.715065
[17:51:52] [DISTANCIA] Posição escrita (navegação): 24.145536
[17:51:53] [CONTROLE] Posição lida (navegação): 5.053784
[17:51:53] [DISTANCIA] Posição escrita (navegação): 98.272110
[17:51:54] [DISTANCIA] Posição escrita (navegação): 64.581703
[17:51:55] [CONTROLE] Posição lida (navegação): 96.715065
[17:51:56] [DISTANCIA] Posição escrita (navegação): 60.326897
[17:51:57] [DISTANCIA] Posição escrita (navegação): 94.607086
[17:51:57] [CONTROLE] Posição lida (navegação): 24.145536
[17:51:58] [DISTANCIA] Posição escrita (navegação): 63.498756
[17:51:59] [DISTANCIA] Posição escrita (navegação): 72.532227
[17:51:59] [CONTROLE] Posição lida (navegação): 98.272110
[17:52:00] [DISTANCIA] Posição escrita (navegação): 8.689487
[17:52:01] [DISTANCIA] Posição escrita (navegação): 33.802132
[17:52:01] [CONTROLE] Posição lida (navegação): 64.581703
[17:52:02] [DISTANCIA] Posição escrita (navegação): 65.608650
[17:52:03] [DISTANCIA] Posição escrita (navegação): 84.797623
[17:52:03] [CONTROLE] Posição lida (navegação): 60.326897
[17:52:04] [DISTANCIA] Posição escrita (navegação): 97.202446
[17:52:05] [DISTANCIA] Posição escrita (navegação): 57.833843
[17:52:06] [CONTROLE] Posição lida (navegação): 94.607086
[17:52:06] [DISTANCIA] Posição escrita (navegação): 1.694813
[17:52:06] [MAIN] Thread 0 finalizada
[17:52:06] [MAIN] Esperando thread 1
[17:52:06] [MAIN] Thread 1 finalizada
[17:52:06] [MAIN] Esperando thread 2
[17:52:06] [MAIN] Thread 2 finalizada
[17:52:06] [MAIN] Esperando thread 3
[17:52:06] [MAIN] Thread 3 finalizada
[17:52:06] [MAIN] Esperando thread 4
[17:52:08] [CONTROLE] Posição lida (navegação): 63.498756
[17:52:10] [CONTROLE] Posição lida (navegação): 72.532227
[17:52:12] [CONTROLE] Posição lida (navegação): 8.689487
[17:52:14] [CONTROLE] Posição lida (navegação): 33.802132
[17:52:16] [CONTROLE] Posição lida (navegação): 65.608650
[17:52:18] [CONTROLE] Posição lida (navegação): 84.797623
[17:52:20] [CONTROLE] Posição lida (navegação): 97.202446
[17:52:22] [CONTROLE] Posição lida (navegação): 57.833843
[17:52:24] [CONTROLE] Posição lida (navegação): 1.694813
[17:52:26] [MAIN] Thread 4 finalizada

```

```
Buffer navegação: 94.6071 63.4988 72.5322 8.68949 33.8021 65.6087 84.7976
↳ 97.2024 57.8338 1.69481
Buffer nível: 53.8312 69.0436 32.9425 31.726 96.7376 42.4353 53.3184 43.8197
↳ 86.9565 38.2572
[17:52:26] [MAIN] Execução finalizada
```

TESTE COM UMA FALHA DETECTADA

```
aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:15:56] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:15:56] [MAIN] Buffers inicializados
[18:15:56] [MAIN] Criando thread distancia_percorrida
[18:15:56] [MAIN] Criando thread inspecao_camera
[18:15:56] [MAIN] Criando thread coletor_dados
[18:15:56] [MAIN] Criando thread reconstrucao_teto
[18:15:56] [COLETOR] Thread inicializada
[18:15:56] [RECONSTRUCAO] Thread inicializada
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [MAIN] Criando thread controle_navegacao
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 74.495483
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 42.898727
[18:15:56] [CONTROLE] Posição lida (navegação): 0.000000
[18:15:56] [MAIN] Esperando thread 0
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.575493
[18:15:56] [COLETOR] Posição lida (nível): 74.495483
[18:15:56] [COLETOR] Posição lida (nível): 42.898727
[18:15:56] [COLETOR] Posição lida (nível): 34.575493
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 98.192505
[18:15:56] [COLETOR] Posição lida (nível): 98.192505
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 15.192101
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 73.814850
[18:15:56] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:15:56] [COLETOR] Posição lida (nível): 15.192101
[18:15:56] [COLETOR] Posição lida (nível): 73.814850
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 97.350990
[18:15:56] [CAMERA] Inspeção iniciada
[18:15:56] [CAMERA] Inspeção finalizada
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 55.099091
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 48.265957
[18:15:56] [COLETOR] Posição lida (nível): 97.350990
[18:15:56] [COLETOR] Posição lida (nível): 55.099091
```

[18:15:56] [COLETOR] Posição lida (nível): 48.265957
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 30.761057
 [18:15:56] [COLETOR] Posição lida (nível): 30.761057
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 49.804440
 [18:15:56] [COLETOR] Posição lida (nível): 49.804440
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 46.795132
 [18:15:56] [COLETOR] Posição lida (nível): 46.795132
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 12.473819
 [18:15:56] [COLETOR] Posição lida (nível): 12.473819
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 68.368179
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.957470
 [18:15:56] [COLETOR] Posição lida (nível): 68.368179
 [18:15:56] [COLETOR] Posição lida (nível): 34.957470
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 90.110184
 [18:15:56] [COLETOR] Posição lida (nível): 90.110184
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 39.241226
 [18:15:56] [COLETOR] Posição lida (nível): 39.241226
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 31.790064
 [18:15:56] [COLETOR] Posição lida (nível): 31.790064
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 93.427567
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 61.848675
 [18:15:56] [RECONSTRUCAO] Thread finalizada
 [18:15:56] [COLETOR] Posição lida (nível): 93.427567
 [18:15:56] [COLETOR] Posição lida (nível): 61.848675
 [18:15:57] [DISTANCIA] Posição escrita (navegação): 62.380260
 [18:15:58] [CONTROLE] Posição lida (navegação): 0.000000
 [18:15:58] [DISTANCIA] Posição escrita (navegação): 51.585777
 [18:15:59] [DISTANCIA] Posição escrita (navegação): 62.972610
 [18:16:00] [CONTROLE] Posição lida (navegação): 62.972610
 [18:16:00] [DISTANCIA] Posição escrita (navegação): 65.160225
 [18:16:01] [DISTANCIA] Posição escrita (navegação): 31.197983
 [18:16:02] [CONTROLE] Posição lida (navegação): 65.160225
 [18:16:02] [DISTANCIA] Posição escrita (navegação): 45.259418
 [18:16:03] [DISTANCIA] Posição escrita (navegação): 81.480118
 [18:16:04] [CONTROLE] Posição lida (navegação): 31.197983
 [18:16:04] [DISTANCIA] Posição escrita (navegação): 19.269875
 [18:16:05] [DISTANCIA] Posição escrita (navegação): 17.381445
 [18:16:06] [CONTROLE] Posição lida (navegação): 45.259418
 [18:16:06] [DISTANCIA] Posição escrita (navegação): 43.342255
 [18:16:07] [DISTANCIA] Posição escrita (navegação): 1.306038
 [18:16:08] [CONTROLE] Posição lida (navegação): 81.480118
 [18:16:08] [DISTANCIA] Posição escrita (navegação): 12.224937

```

[18:16:09] [DISTANCIA] Posição escrita (navegação): 75.290260
[18:16:10] [CONTROLE] Posição lida (navegação): 19.269875
[18:16:10] [DISTANCIA] Posição escrita (navegação): 55.725075
[18:16:11] [DISTANCIA] Posição escrita (navegação): 83.067741
[18:16:12] [CONTROLE] Posição lida (navegação): 17.381445
[18:16:12] [DISTANCIA] Posição escrita (navegação): 20.277987
[18:16:13] [DISTANCIA] Posição escrita (navegação): 60.572338
[18:16:14] [CONTROLE] Posição lida (navegação): 43.342255
[18:16:14] [DISTANCIA] Posição escrita (navegação): 44.572971
[18:16:15] [DISTANCIA] Posição escrita (navegação): 38.374176
[18:16:16] [CONTROLE] Posição lida (navegação): 1.306038
[18:16:16] [DISTANCIA] Posição escrita (navegação): 34.950138
[18:16:16] [MAIN] Thread 0 finalizada
[18:16:16] [MAIN] Esperando thread 1
[18:16:16] [MAIN] Thread 1 finalizada
[18:16:16] [MAIN] Esperando thread 2
[18:16:16] [MAIN] Thread 2 finalizada
[18:16:16] [MAIN] Esperando thread 3
[18:16:16] [MAIN] Thread 3 finalizada
[18:16:16] [MAIN] Esperando thread 4
[18:16:18] [CONTROLE] Posição lida (navegação): 12.224937
[18:16:20] [CONTROLE] Posição lida (navegação): 75.290260
[18:16:22] [CONTROLE] Posição lida (navegação): 55.725075
[18:16:24] [CONTROLE] Posição lida (navegação): 83.067741
[18:16:26] [CONTROLE] Posição lida (navegação): 20.277987
[18:16:28] [CONTROLE] Posição lida (navegação): 60.572338
[18:16:30] [CONTROLE] Posição lida (navegação): 44.572971
[18:16:32] [CONTROLE] Posição lida (navegação): 38.374176
[18:16:34] [CONTROLE] Posição lida (navegação): 34.950138
[18:16:36] [MAIN] Thread 4 finalizada
Buffer navegação: 1.30604 12.2249 75.2903 55.7251 83.0677 20.278 60.5723
↪ 44.573 38.3742 34.9501
Buffer nível: 49.8044 46.7951 12.4738 68.3682 34.9575 90.1102 39.2412 31.7901
↪ 93.4276 61.8487
[18:16:36] [MAIN] Execução finalizada

```

TESTE COM MÚLTIPLAS FALHAS

```

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:14:34] [PROCESS0] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:14:34] [MAIN] Buffers inicializados
[18:14:34] [MAIN] Criando thread distancia_percorrida
[18:14:34] [MAIN] Criando thread inspecao_camera
[18:14:34] [MAIN] Criando thread coletor_dados

```

```

[18:14:34] [MAIN] Criando thread reconstrucao_teto
[18:14:34] [COLETOR] Thread inicializada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Thread inicializada
[18:14:34] [MAIN] Criando thread controle_navegacao
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 11.893382
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [MAIN] Esperando thread 0
[18:14:34] [COLETOR] Posição lida (nível): 11.893382
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 33.125107
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 33.125107
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.220352
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 70.220352
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 38.717987
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 38.717987
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 89.701645
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 89.701645
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 17.111982
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 17.111982
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 85.353203

```

[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 85.353203
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 42.381588
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Posição lida (nível): 42.381588
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 18.459126
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 18.459126
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 78.904755
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 78.904755
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 46.468765
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 46.468765
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.231094
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 68.231094
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.282280
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 70.282280
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 47.991817
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida


```

[18:14:34] [COLETOR] Posição lida (nível): 47.991817
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 55.565323
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 55.565323
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 76.516632
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 76.516632
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 59.348728
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 59.348728
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 8.146796
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 8.146796
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.844254
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 68.844254
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Thread finalizada
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:35] [DISTANCIA] Posição escrita (navegação): 93.736534
[18:14:36] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[18:14:36] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:36] [DISTANCIA] Escritor retomou execução
[18:14:36] [DISTANCIA] Posição escrita (navegação): 80.958572

```

```

[18:14:37] [DISTANCIA] Posição escrita (navegação): 35.029560
[18:14:38] [CONTROLE] Posição lida (navegação): 35.029560
[18:14:38] [DISTANCIA] Posição escrita (navegação): 91.344322
[18:14:39] [DISTANCIA] Posição escrita (navegação): 96.001343
[18:14:40] [CONTROLE] Posição lida (navegação): 91.344322
[18:14:40] [DISTANCIA] Posição escrita (navegação): 11.806067
[18:14:41] [DISTANCIA] Posição escrita (navegação): 73.489479
[18:14:42] [CONTROLE] Posição lida (navegação): 96.001343
[18:14:42] [DISTANCIA] Posição escrita (navegação): 10.448226
[18:14:43] [DISTANCIA] Posição escrita (navegação): 62.412590
[18:14:44] [CONTROLE] Posição lida (navegação): 11.806067
[18:14:44] [DISTANCIA] Posição escrita (navegação): 64.524155
[18:14:45] [DISTANCIA] Posição escrita (navegação): 19.370922
[18:14:46] [CONTROLE] Posição lida (navegação): 73.489479
[18:14:46] [DISTANCIA] Posição escrita (navegação): 46.210743
[18:14:47] [DISTANCIA] Posição escrita (navegação): 97.758255
[18:14:48] [CONTROLE] Posição lida (navegação): 10.448226
[18:14:48] [DISTANCIA] Posição escrita (navegação): 80.271263
[18:14:49] [DISTANCIA] Posição escrita (navegação): 34.450363
[18:14:50] [CONTROLE] Posição lida (navegação): 62.412590
[18:14:50] [DISTANCIA] Posição escrita (navegação): 30.559107
[18:14:51] [DISTANCIA] Posição escrita (navegação): 68.277695
[18:14:52] [CONTROLE] Posição lida (navegação): 64.524155
[18:14:52] [DISTANCIA] Posição escrita (navegação): 99.241768
[18:14:53] [DISTANCIA] Posição escrita (navegação): 39.216423
[18:14:54] [CONTROLE] Posição lida (navegação): 19.370922
[18:14:54] [DISTANCIA] Posição escrita (navegação): 28.282803
[18:14:54] [MAIN] Thread 0 finalizada
[18:14:54] [MAIN] Esperando thread 1
[18:14:54] [MAIN] Thread 1 finalizada
[18:14:54] [MAIN] Esperando thread 2
[18:14:54] [MAIN] Thread 2 finalizada
[18:14:54] [MAIN] Esperando thread 3
[18:14:54] [MAIN] Thread 3 finalizada
[18:14:54] [MAIN] Esperando thread 4
[18:14:56] [CONTROLE] Posição lida (navegação): 46.210743
[18:14:58] [CONTROLE] Posição lida (navegação): 97.758255
[18:15:00] [CONTROLE] Posição lida (navegação): 80.271263
[18:15:02] [CONTROLE] Posição lida (navegação): 34.450363
[18:15:04] [CONTROLE] Posição lida (navegação): 30.559107
[18:15:06] [CONTROLE] Posição lida (navegação): 68.277695
[18:15:08] [CONTROLE] Posição lida (navegação): 99.241768
[18:15:10] [CONTROLE] Posição lida (navegação): 39.216423
[18:15:12] [CONTROLE] Posição lida (navegação): 28.282803
[18:15:14] [MAIN] Thread 4 finalizada
Buffer navegação: 19.3709 46.2107 97.7583 80.2713 34.4504 30.5591 68.2777
↪ 99.2418 39.2164 28.2828
Buffer nível: 46.4688 68.2311 70.2823 47.9918 55.5653 76.5166 59.3487 8.1468
↪ 68.8443 26.9246
[18:15:14] [MAIN] Execução finalizada

```

```

[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Posição escrita (navegação): 24.064884
[19:06:52] [RECONSTRUCAO] Thread inicializada
[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados

```

Figura 6: Teste de Buffer de Nível Vazio

```

[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 112.025932 | Ocupação: 1/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 160.867523 | Ocupação: 2/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 122.139771 | Ocupação: 3/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 86.595070 | Ocupação: 4/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 146.985657 | Ocupação: 5/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 39.833908 | Ocupação: 6/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 103.447708 | Ocupação: 7/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 92.925583 | Ocupação: 8/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 125.545197 | Ocupação: 9/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 124.471046 | Ocupação: 10/10
[19:13:01] [RECONSTRUCAO] Buffer de nível cheio -> produtor aguardando espaço
[19:13:01] [CONTROLE] Leitor retomou execução
[19:13:01] [DISTANCIA] Posição escrita (navegação): 71.950806
[19:13:01] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:13:01] [COLETOR] Posição lida (nível): 112.025932
[19:13:01] [COLETOR] Posição lida (nível): 160.867523
[19:13:01] [COLETOR] Posição lida (nível): 122.139771
[19:13:01] [COLETOR] Posição lida (nível): 86.595070
[19:13:01] [COLETOR] Posição lida (nível): 146.985657
[19:13:01] [COLETOR] Posição lida (nível): 39.833908
[19:13:01] [COLETOR] Posição lida (nível): 103.447708
[19:13:01] [COLETOR] Posição lida (nível): 92.925583
[19:13:01] [COLETOR] Posição lida (nível): 125.545197
[19:13:01] [COLETOR] Posição lida (nível): 124.471046
[19:13:01] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:13:01] [RECONSTRUCAO] Espaço do buffer de nível liberado -> produtor retomou execução
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 52.224766 | Ocupação: 1/10

```

Figura 7: Teste de Buffer de Nível Cheio

```

[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada
[19:12:47] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:12:47] [RECONSTRUCAO] Thread inicializada
[19:12:47] [RECONSTRUCAO] Leitor aguardando acesso ao buffer navegacao
[19:12:47] [MAIN] Criando thread controle_navegacao
[19:12:47] [MAIN] Esperando thread 0
[19:12:47] [CONTROLE] Leitor aguardando acesso ao buffer navegacao
[19:12:48] [DISTANCIA] Posição escrita (navegação): 73.074905
[19:12:48] [RECONSTRUCAO] Leitor retomou execução
[19:12:48] [CONTROLE] Leitor retomou execução

```

Figura 8: Teste de Bloqueio de Escritores

```

[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Posição lida (navegação): 53.040310
[19:06:53] [DISTANCIA] Escritor retomou execução
[19:06:53] [DISTANCIA] Posição escrita (navegação): 99.067177
[19:06:53] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO

```

Figura 9: Teste de Bloqueio de Leitores

```

[19:13:18] [MAIN] Thread 0 finalizada
[19:13:18] [MAIN] Esperando thread 1
[19:13:18] [MAIN] Thread 1 finalizada
[19:13:18] [MAIN] Esperando thread 2
[19:13:18] [MAIN] Thread 2 finalizada
[19:13:18] [MAIN] Esperando thread 3
[19:13:18] [MAIN] Thread 3 finalizada
[19:13:18] [MAIN] Esperando thread 4
[19:13:19] [CONTROLE] Posição lida (navegação): 97.285744
[19:13:20] [CONTROLE] Posição lida (navegação): 12.477365
[19:13:21] [CONTROLE] Posição lida (navegação): 46.973194
[19:13:22] [CONTROLE] Posição lida (navegação): 20.479900
[19:13:23] [CONTROLE] Posição lida (navegação): 82.494011
[19:13:24] [CONTROLE] Posição lida (navegação): 25.792789
[19:13:24] [MAIN] Thread 4 finalizada
Buffer navegacao: 14.1857 37.1402 80.635 85.4882 97.2857 12.4774 46.9732 20.4799 82.494 25.7928
Buffer nível: 146.986 39.8339 103.448 92.9256 125.545 124.471 52.2248 108.817 163.728 65.7006
[19:13:24] [MAIN] Execução finalizada

```

Figura 10: Teste de Finalização das Threads

5.2 includes.hpp

```

1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <chrono>
6  #include <cstdlib>
7  #include <cstring>
8  #include <csignal>
9  #include <random>
10
11 #include <thread>
12 #include <atomic>
13 #include <mutex>
14 #include <condition_variable>
15 #include <semaphore>
16 #include <pthread.h>
17 #include <iomanip>

```

```

18
19 #include <unistd.h>
20 #include <sys/ipc.h>
21 #include <sys/shm.h>
22 #include <sys/wait.h>
23
24 #include <functional>
25 #include <boost/asio.hpp>
26 #include <sys/wait.h>
27
28 #include <boost/interprocess/file_mapping.hpp>
29 #include <boost/interprocess/mapped_region.hpp>
30 #include <fstream>
31 #include <cstdio>
32 #include <algorithm>
33 #include <cmath>
34
35 // MQTT
36 #include "mqtt_client.hpp"
37 #include "mqtt_publisher.hpp"
38 #include "mqtt_subscriber.hpp"
39 #include "mqtt_manager.hpp"
40 #include "mqtt_config.hpp"

```

5.3 main.cpp

```

1 #include "include/includes.hpp"
2 #include "include/shared_memory.hpp"
3 #include "include/sincronizacao.hpp"
4 #include "include/simulacao.hpp"
5
6 int main (){
7
8     using namespace boost::interprocess;
9
10    simulacao();
11
12    std::remove(SHM_FILE); // remove a memória compartilhada caso ela já
    ↳ exista
13
14    {
15        std::filebuf fbuf;
16        fbuf.open(SHM_FILE, std::ios_base::in | std::ios_base::out |
    ↳ std::ios_base::trunc | std::ios_base::binary);
17
18        if(!fbuf.is_open()){
19            std::cerr << "Erro ao criar arquivo de memória compartilhada" <<
    ↳ std::endl;
20            return 1;
21        }
22

```

```

23     fbuf.pubseekoff(sizeof(MemoriaCompartilhada)-1, std::ios_base::beg);
24     fbuf.putc(0);
25     fbuf.close();
26 } // cria arquivo que será mapeado na memória
27
28 std::cout << "Arquivo de memória criado em: " << SHM_FILE << std::endl;
29 std::cout << "Tamanho da struct C++: " << sizeof(MemoriaCompartilhada) << "
    ↳ bytes" << std::endl;
30
31 file_mapping shm_file(SHM_FILE, read_write);
32 mapped_region region(shm_file, read_write); // mapeia o arquivo no espaço
    ↳ de memória
33
34 MemoriaCompartilhada* shm =
    ↳ static_cast<MemoriaCompartilhada*>(region.get_address());
35
36 // Inicializa os valores da memória compartilhada
37 shm->i_encoder = false;
38 shm->i_lidar = 0;
39
40 shm->o_liga_camera = false;
41 shm->o_aceleracao = 0;
42
43 shm->e_inspecao = false;
44 shm->e_automatico = false;
45
46 shm->c_automatico = false;
47 shm->c_man = false;
48 shm->j_sp_velocidade = 0;
49
50 shm->c_encerrar = false;
51
52 shm->variacao_severa = 10.0f; // limiar padrão (configurável via MQTT)
53 shm->posicao_x = 0.0f;
54 shm->perfil_y = 0.0f;
55 shm->perfil_confianca = 0.0f;
56 shm->perfil_novo = false;
57
58
59 // CRIAÇÃO DE PROCESSOS (FORK)
60
61 // PROCESSO 1: SIMULAÇÃO (interface.py usa shared memory)
62 pid_t pid_interface = fork();
63 if (pid_interface == 0) {
64     log_message("PROCESSO", "Processo de simulação Pygame criado");
65     execlp("python3", "python3", "src/interface.py", nullptr);
66     perror("Erro ao executar interface.py");
67     exit(1);
68 } else if (pid_interface < 0) {
69     perror("Erro ao criar processo da interface");
70     exit(1);

```

```

71     }
72
73     //PROCESSO 2: OPERAÇÃO REMOTA (interface_mqtt.py usa MQTT)
74     pid_t pid_remoto = fork();
75     if (pid_remoto == 0) {
76         log_message("PROCESSO", "Processo de operação remota MQTT criado");
77         execlp("python3", "python3", "src/interface_mqtt.py", nullptr);
78         perror("Erro ao executar interface_mqtt.py");
79         exit(1);
80     } else if (pid_remoto < 0) {
81         perror("Erro ao criar processo de operação remota");
82         exit(1);
83     }
84
85
86     // PROCESSO 2: COMANDO (FILHO) E CONTROLE (PAI)
87     pid_t pid_controle = fork();
88
89     if (pid_controle == 0) {
90         // BLOCO DO FILHO: COMANDO DE NAVEGAÇÃO
91         log_message("PROCESSO", "Processo comando_navegacao criado");
92
93         file_mapping shm_child(SHM_FILE, read_write); // abre a memória
94         ↪ compartilhada criada
95
96         mapped_region region_child(shm_child, read_write);
97         MemoriaCompartilhada* shm_filho =
98         ↪ static_cast<MemoriaCompartilhada*>(region_child.get_address());
99
100        // Instanciar o Asio APENAS para o filho
101        boost::asio::io_context io_filho;
102        boost::asio::io_context::strand strand_filho(io_filho);
103
104        tempo_tarefa t_comando(io_filho, milisegundos(500));
105        t_comando.async_wait(boost::asio::bind_executor(strand_filho,
106            std::bind(comando_navegacao, std::placeholders::_1, &t_comando,
107                ↪ &strand_filho, shm)));
108
109        shm->c_automatico = true;
110        shm->j_sp_velocidade = 50;
111
112        std::cout << "Comando de navegação escreveu na memória compartilhada:" <<
113        ↪ std::endl;
114        std::cout << "c_automatico = " << shm_filho->c_automatico <<
115        ↪ std::endl;
116        std::cout << "j_sp_velocidade = " << shm_filho->j_sp_velocidade <<
117        ↪ std::endl;
118
119        // Iniciar o laço de eventos do filho
120        io_filho.run();
121    }

```



```

116     exit(0);
117 }
118 else if (pid_controle < 0) {
119     perror("Erro ao criar processo de comando\n");
120     exit(1);
121 }
122 else {
123     // BLOCO DO PAI: CONTROLE DE NAVEGAÇÃO E SENSORIAMENTO
124     log_message("PROCESSO", "Processo de controle de navegação iniciado");
125
126     std::mutex shm_mutex;
127
128     MQTTManager mqtt_manager(shm, shm_mutex);
129
130     log_message("MAIN", "Iniciando MQTT Manager...");
131     if (mqtt_manager.initialize()) {
132         log_message("MAIN", "MQTT Manager inicializado com sucesso");
133     } else {
134         log_message("MAIN", "Falha ao inicializar MQTT Manager - continuando
        ↳ sem MQTT");
135     }
136
137     boost::asio::io_context io_pai;
138
139     boost::asio::io_context::strand strand_navegacao(io_pai); // Para dist e
        ↳ controle
140     boost::asio::io_context::strand strand_processamento(io_pai); // Para
        ↳ reconstrução e dados
141     boost::asio::io_context::strand strand_camera(io_pai); // Para câmera
        ↳ isolada
142
143     struct VarCondSinc sinc;
144
145     // DEFINIÇÃO DOS TEMPORIZADORES
146     tempo_tarefa t1_dist(io_pai, milisegundos(PERODO_DISTANCIA));
147     tempo_tarefa t2_cam(io_pai, milisegundos(PERODO_CAMERA));
148     tempo_tarefa t3_col(io_pai, milisegundos(PERODO_COLETOR));
149     tempo_tarefa t4_rec(io_pai, milisegundos(PERODO_RECONSTRUCAO));
150     tempo_tarefa t5_con(io_pai, milisegundos(PERODO_CONTROLE));
151
152     // CRIAÇÃO DE SIGNAL
153     boost::asio::signal_set sinais (io_pai);
154     sinais.add(SIGUSR1);
155
156     // AGENDAMENTO DAS TAREFAS (ASYNC_WAIT)
157     log_message("MAIN", "Agendando tarefas assíncronas...");
158
159     // Associando à Strand de Navegação (Crítica)
160     t1_dist.async_wait(boost::asio::bind_executor(strand_navegacao,
161         std::bind(distancia_percorrida, std::placeholders::_1, &t1_dist,
        ↳ &strand_navegacao, shm, std::ref(sinc))));

```



```

162
163     t5_con.async_wait(boost::asio::bind_executor(strand_navegacao,
164         std::bind(controle_navegacao, std::placeholders::_1, &t5_con,
165             ↪ &strand_navegacao, shm, std::ref(sinc))));
166
167     // Associando à Strand de Processamento (Menos Crítica)
168     t4_rec.async_wait(boost::asio::bind_executor(strand_processamento,
169         std::bind(reconstrucao_teto, std::placeholders::_1, &t4_rec,
170             ↪ &strand_processamento, shm, std::ref(sinc))));
171
172     t3_col.async_wait(boost::asio::bind_executor(strand_processamento,
173         std::bind(coletor_dados, std::placeholders::_1, &t3_col,
174             ↪ &strand_processamento, shm, std::ref(sinc))));
175
176     sinais.async_wait(
177         std::bind(handler_signal, std::placeholders::_1, std::placeholders::_2,
178             ↪ &t2_cam, &strand_camera,
179             std::ref(sinc.mutex_camera), shm, &sinais));
180
181     // EXECUÇÃO (THREAD POOL)
182     // Reativar o Pool de Threads. Com 4 threads, as nossas 3 Strands podem
183     ↪ rodar
184     // simultaneamente em núcleos reais do processador.
185
186     while (!shm->c_automatico) {
187         std::this_thread::sleep_for(std::chrono::milliseconds(5));
188     }
189
190     int num_threads = 4;
191     std::vector<std::thread> thread_pool;
192     log_message("MAIN", "Iniciando Thread Pool do Boost.Asio com " +
193         ↪ std::to_string(num_threads) + " threads");
194
195     for (int i = 0; i < num_threads; ++i) {
196         thread_pool.emplace_back([&io_pai]() { io_pai.run(); });
197     }
198
199     std::cout << "Sistema rodando. Feche a janela de simulação para
200     ↪ encerrar.\n";
201
202     // Aguarda até que a interface de simulação sinalize o encerramento (janela
203     ↪ fechada)
204     while (!shm->c_encerrar) {
205         std::this_thread::sleep_for(std::chrono::milliseconds(200));
206     }
207
208     std::cout << "Sinal de encerramento recebido. Desligando tarefas...\n";
209
210     io_pai.stop();
211
212     //SINCRONIZAÇÃO FINAL

```

```

205     for (auto& t : thread_pool) {
206         if (t.joinable()) {
207             std::cout << "Esperando join da thread...\n";
208             t.join();
209         }
210     }
211
212     std::cout << "Thread pool encerrada.\n";
213
214     // Espera o processo filho acabar
215     waitpid(pid_controle, nullptr, 0);
216
217     // Finalizar MQTT Manager
218     log_message("MAIN", "Finalizando MQTT Manager...");
219     mqtt_manager.shutdown();
220     log_message("MAIN", "MQTT Manager finalizado");
221
222     log_message("MAIN", "Execução finalizada. Limpando memória.");
223
224     // Desanexa memória no pai
225     std::remove(SHM_FILE);
226 }
227 }

```

5.4 sincronizacao.cpp

```

1  #include "sincronizacao.hpp"
2
3  #define ELEMENTOS_BUFFERS 10
4
5  //sincronização
6  std::condition_variable camera;
7
8  //teste para finalizar a camera
9  bool finalizar_camera = false;
10 int eventos_camera = 0;
11
12 //Variaveis de condição buffers
13 std::condition_variable leitura_buffer_navegacao;
14 std::condition_variable escrita_buffer_navegacao;
15
16 int AR_NAVEGACAO = 0;
17 int WR_NAVEGACAO = 0;
18 int AW_NAVEGACAO = 0;
19 int WW_NAVEGACAO = 0;
20
21 std::condition_variable leitura_buffer_nivel;
22 std::condition_variable escrita_buffer_nivel;
23 int dados_nivel = 0;
24
25 //Funções auxiliares de debug

```

```

26  std::mutex mutex_log;
27  int miss[4] = {0, 0, 0, 0};
28  int executado[4] = {0, 0, 0, 0};
29
30
31  // Estimativa de velocidade compartilhada entre distancia_percorrida e
    ↳ controle_navegacao.
32  // Ambas as tarefas rodam na mesma strand (strand_navegacao), portanto sem
    ↳ corrida de dados.
33  static float s_vel_encoder_ms = 0.0f;
34  static auto s_t_ultimo_encoder = std::chrono::steady_clock::now();
35  static bool s_encoder_ativo = false;
36
37  void log_message(const std::string& thread, const std::string& mensagem) {
38      std::lock_guard<std::mutex> lock(mutex_log);
39      auto agora = std::chrono::system_clock::now();
40      auto tempo = std::chrono::system_clock::to_time_t(agora);
41      std::cout << "[" << std::put_time(std::localtime(&tempo), "%H:%M:%S") << "]"
    ↳ "
42          << "[" << thread << "]" << mensagem << std::endl;
43  }
44
45  float numero_aleatorio_debugg() {
46      std::random_device rd;
47      std::mt19937 gen(rd());
48      std::uniform_real_distribution<float> dis(1.0f, 100.0f);
49      return dis(gen);
50  }
51
52
53  // FUNÇÃO AUXILIAR DE TEMPO REAL
54  void reagendar_tarefa(boost::asio::steady_timer* t, int periodo_us, const
    ↳ std::string& nome_tarefa) {
55      auto agora = std::chrono::steady_clock::now();
56      auto atraso = std::chrono::duration_cast<std::chrono::milliseconds>(agora -
    ↳ t->expiry()).count();
57      if (atraso > 5000) {
58          log_message(nome_tarefa, "ALERTA: Deadline violado! Atraso de " +
    ↳ std::to_string(atraso) + " ms");
59      }
60
61      auto proximo_ciclo = t->expiry() +
    ↳ boost::asio::chrono::milliseconds(periodo_us);
62
63      if (proximo_ciclo < agora) {
64          proximo_ciclo = agora + boost::asio::chrono::milliseconds(periodo_us);
65      }
66
67      t->expires_at(proximo_ciclo);
68  }
69

```

```

70 void handler_signal(const boost::system::error_code& error, int signal_number,
71                    boost::asio::steady_timer* t,
72                    ↪ boost::asio::io_context::strand* strand_camera,
73                    std::mutex& mtx, MemoriaCompartilhada* shm,
74                    ↪ boost::asio::signal_set* sinais) {
75
76     if(error) return;
77
78     if (signal_number == SIGUSR1) {
79         eventos_camera++;
80
81         boost::asio::post(*strand_camera, std::bind(inspecao_camera,
82             boost::system::error_code(), t, strand_camera,
83             ↪ std::ref(mtx), shm));
84     }
85
86     //REAGENDAMENTO
87
88     sinais->async_wait(std::bind(handler_signal, std::placeholders::_1,
89         ↪ std::placeholders::_2,
90         t, strand_camera, std::ref(mtx), shm,
91         ↪ sinais));
92 }
93
94 // TAREFAS ASSÍNCRONAS DO SISTEMA
95 void comando_navegacao(const boost::system::error_code& e,
96 ↪ boost::asio::steady_timer* t,
97 ↪ boost::asio::io_context::strand* strand,
98 ↪ MemoriaCompartilhada* shm)
99 {
100     if (e) return;
101     if (shm->c_encerrar) {
102         log_message("COMANDO", "Encerrando comando de navegação.");
103         return;
104     }
105
106     // Traduz comandos em estados | manual/automático
107     if (shm->c_automatico) {
108         shm->e_automatico = true;
109         shm->c_automatico = false;
110         log_message("COMANDO", "Modo AUTOMÁTICO ativado");
111     }
112     if (shm->c_man) {
113         shm->e_automatico = false;
114         shm->c_man = false;
115         shm->o_aceleracao = 0; // cancela qualquer desaceleração de inspeção
116         log_message("COMANDO", "Modo MANUAL ativado");
117     }
118
119     reagendar_tarefa(t, PERIODO_COMANDO, "COMANDO");

```

```

114     t->async_wait(boost::asio::bind_executor(*strand,
115         std::bind(comando_navegacao, std::placeholders::_1, t, strand, shm)));
116 }
117
118
119 void controle_navegacao(const boost::system::error_code& e,
120     ↪ boost::asio::steady_timer* t,
121     boost::asio::io_context::strand* strand,
122     ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
123 {
124     if (e) return;
125     if (shm->c_encerrar) return;
126
127     bool processou_algo = false;
128
129     {
130         std::lock_guard<std::mutex> lock(sinc.mtx_navegacao);
131         while (sinc.disp_naveg_c > 0) {
132             sinc.LER_IDX_NAVEG_C = (sinc.LER_IDX_NAVEG_C + 1) %
133                 ↪ ELEMENTOS_BUFFERS;
134             sinc.disp_naveg_c--;
135             processou_algo = true;
136         }
137     }
138
139     if (processou_algo) {
140         executado[0]++;
141     } else {
142         miss[0]++;
143     }
144
145     // Controlador PID de velocidade
146     // Computa P, I e D a partir do encoder. Em um sistema embarcado real a
147     // saída acionaria o driver de motor; aqui a simulação Python já gerencia
148     ↪ a
149     // física da velocidade, então a saída é calculada mas não sobrescreve
150     // o_aceleracao (que é exclusivo de reconstrucao_teto para inspeção).
151     if (shm->e_automatico && s_encoder_ativo) {
152         static float integral = 0.0f;
153         static float erro_ant = 0.0f;
154         static bool insp_ant = false;
155
156         constexpr float DT = PERIODO_CONTROLE / 1000.0f;
157         constexpr float Kp = 5.0f, Ki = 1.0f, Kd = 0.2f;
158
159         // Zera integrador ao sair de inspeção para evitar windup residual
160         if (insp_ant && !shm->e_inspecao) { integral = 0.0f; erro_ant = 0.0f; }
161         insp_ant = shm->e_inspecao;
162
163         float sp_ms = shm->j_sp_velocidade * 0.04f; // sp=50 → 2.0 m/s
164         float erro = sp_ms - s_vel_encoder_ms;

```

```

161
162     integral = std::clamp(integral + erro * DT, -20.0f, 20.0f);
163     float derivada = (erro - erro_ant) / DT;
164     erro_ant = erro;
165
166     [[maybe_unused]] float saida_pid =
167         std::clamp(Kp * erro + Ki * integral + Kd * derivada, -100.0f,
168             ↪ 100.0f);
169
170 }
171
172 reagendar_tarefa(t, PERIODO_CONTROLE, "CONTROLE");
173 t->async_wait(boost::asio::bind_executor(*strand,
174     std::bind(controle_navegacao, std::placeholders::_1, t, strand, shm,
175         ↪ std::ref(sinc))));
176
177 void distancia_percorrida(const boost::system::error_code& e,
178     ↪ boost::asio::steady_timer* t,
179     boost::asio::io_context::strand* strand,
180     ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
181 {
182     if (e) return;
183     if (shm->c_encerrar) return;
184
185     // Detecta transição do encoder e acumula distância em metros
186     static bool ultimo_encoder = false;
187     bool encoder_atual = shm->i_encoder;
188     auto agora_enc = std::chrono::steady_clock::now();
189
190     if (encoder_atual != ultimo_encoder) {
191         // Nova transição: calcula velocidade = 1 metro / tempo desde última
192         ↪ transição
193         if (s_encoder_ativo) {
194             float dt = std::chrono::duration<float>(agora_enc -
195                 ↪ s_t_ultimo_encoder).count();
196             if (dt > 0.01f)
197                 s_vel_encoder_ms = 1.0f / dt;
198         }
199         s_t_ultimo_encoder = agora_enc;
200         s_encoder_ativo = true;
201         shm->posicao_x += 1.0f;
202         ultimo_encoder = encoder_atual;
203     } else if (s_encoder_ativo) {
204         // Entre transições: velocidade estimada decai naturalmente com o
205         ↪ tempo
206         float dt = std::chrono::duration<float>(agora_enc -
207             ↪ s_t_ultimo_encoder).count();
208         if (dt > 0.01f)
209             s_vel_encoder_ms = 1.0f / dt;

```

```

204     }
205
206     {
207         std::lock_guard<std::mutex> lock(sinc.mtx_navegacao);
208
209         if (sinc.disp_naveg_c >= ELEMENTOS_BUFFERS) {
210             sinc.LER_IDX_NAVEG_C = (sinc.LER_IDX_NAVEG_C + 1) %
                ↪ ELEMENTOS_BUFFERS;
211             sinc.disp_naveg_c--;
212         }
213         if (sinc.disp_naveg_r >= ELEMENTOS_BUFFERS) {
214             sinc.LER_IDX_NAVEG_R = (sinc.LER_IDX_NAVEG_R + 1) %
                ↪ ELEMENTOS_BUFFERS;
215             sinc.disp_naveg_r--;
216         }
217
218         // Escreve a posição atual para controle e reconstrução
219         sinc.BUFFER_NAVEGACAO[sinc.ESC_IDX_NAVEG] = shm->posicao_x;
220         sinc.ESC_IDX_NAVEG = (sinc.ESC_IDX_NAVEG + 1) % ELEMENTOS_BUFFERS;
221
222         sinc.disp_naveg_c++;
223         sinc.disp_naveg_r++;
224     }
225
226     executado[1]++;
227
228     reagendar_tarefa(t, PERIODO_DISTANCIA, "DISTANCIA");
229     t->async_wait(boost::asio::bind_executor(*strand,
230         std::bind(distancia_percorrida, std::placeholders::_1, t, strand, shm,
                ↪ std::ref(sinc))));
231 }
232
233
234 void reconstrucao_teto(const boost::system::error_code& e,
    ↪ boost::asio::steady_timer* t,
235         boost::asio::io_context::strand* strand,
    ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
236 {
237     if (e) return;
238     if (shm->c_encerrar) return;
239
240     // Consome atualizações de posição do buffer de navegação
241     bool houve_atualizacao = false;
242     {
243         std::lock_guard<std::mutex> lock_nav(sinc.mtx_navegacao);
244         while (sinc.disp_naveg_r > 0) {
245             sinc.LER_IDX_NAVEG_R = (sinc.LER_IDX_NAVEG_R + 1) %
                ↪ ELEMENTOS_BUFFERS;
246             sinc.disp_naveg_r--;
247             houve_atualizacao = true;
248         }

```

```

249     }
250
251     if (houve_atualizacao) {
252         // Filtro de média móvel (janela de 2 amostras) sobre o LIDAR
253         static float janela[2] = {};
254         static int idx_janela = 0;
255         static bool janela_cheia = false;
256
257         janela[idx_janela % 2] = static_cast<float>(shm->i_lidar);
258         idx_janela++;
259         if (idx_janela >= 2) janela_cheia = true;
260
261         int n = janela_cheia ? 2 : idx_janela;
262         float soma = 0.0f;
263         for (int i = 0; i < n; ++i) soma += janela[i];
264         float media = soma / n;
265
266         // Escreve valor filtrado no buffer de nível
267         {
268             std::lock_guard<std::mutex> lock_nivel(sinc.mtx_nivel);
269
270             // Estabelece baseline a partir das primeiras 15 leituras (antes de
271             // ↳ qualquer anomalia)
272             // Depois compara cada leitura contra esse baseline para detectar
273             // ↳ variação severa
274             static float baseline = 0.0f;
275             static float baseline_soma = 0.0f;
276             static int baseline_n = 0;
277             const int BASELINE_AMOSTRAS = 15;
278
279             if (baseline_n < BASELINE_AMOSTRAS) {
280                 // Aguarda janela cheia e média estabilizada (> 100) para
281                 // ↳ excluir zeros de startup
282                 // e valores de transição como (0+0+0+0+145)/5 = 29 que
283                 // ↳ corrompem o baseline
284                 if (janela_cheia && media > 100.0f) {
285                     baseline_soma += media;
286                     baseline_n++;
287                     if (baseline_n == BASELINE_AMOSTRAS) {
288                         baseline = baseline_soma / BASELINE_AMOSTRAS;
289                         log_message("RECONSTRUCAO", "Baseline estabelecido: " +
290                                 ↳ std::to_string(baseline));
291                     }
292                 }
293             } else {
294                 float variacao = std::abs(media - baseline);
295                 if (variacao > shm->variacao_severa) {
296                     if (!shm->e_inspecao) { // dispara apenas uma vez por
297                         ↳ anomalia
298                         shm->e_inspecao = true;
299                         shm->o_liga_camera = true;
300                     }
301                 }
302             }
303         }
304     }

```



```

294         if (shm->e_automatgico)    // desacelera apenas em modo
        ↪ automático
295             shm->o_aceleracao = -30;
296         kill(getpid(), SIGUSR1);
297         log_message("RECONSTRUCAO", "Variacao severa detectada:
        ↪ " + std::to_string(variacao));
298     }
299     } else if (shm->e_inspecao) { // anomalia superada: restaura
        ↪ estado normal
300         shm->e_inspecao = false;
301         shm->o_liga_camera = false;
302         shm->o_aceleracao = 0;
303         log_message("RECONSTRUCAO", "Anomalia superada, retomando
        ↪ velocidade normal");
304     }
305 }
306
307 if (sinc.disp_nivel >= ELEMENTOS_BUFFERS) {
308     sinc.LER_IDX_NIVEL = (sinc.LER_IDX_NIVEL + 1) %
        ↪ ELEMENTOS_BUFFERS;
309     sinc.disp_nivel--;
310 }
311 sinc.BUFFER_NIVEL[sinc.ESC_IDX_NIVEL] = media;
312 sinc.ESC_IDX_NIVEL = (sinc.ESC_IDX_NIVEL + 1) % ELEMENTOS_BUFFERS;
313 sinc.disp_nivel++;
314
315 // Sinaliza para o publisher MQTT que há novo ponto de perfil
316 shm->perfil_y = media;
317 shm->perfil_novo = true;
318 }
319
320 executado[2]++;
321 } else {
322     miss[2]++;
323 }
324
325 reagendar_tarefa(t, PERIODO_RECONSTRUCAO, "RECONSTRUCAO");
326 t->async_wait(boost::asio::bind_executor(*strand,
327     std::bind(reconstrucao_teto, std::placeholders::_1, t, strand, shm,
        ↪ std::ref(sinc))));
328 }
329
330
331 void coletor_dados(const boost::system::error_code& e,
        ↪ boost::asio::steady_timer* t,
332     boost::asio::io_context::strand* strand,
        ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
333 {
334     static FILE* arquivo_coletor = nullptr;
335     static bool arquivo_aberto = false;
336     static std::size_t contador_coletor = 0;

```

```

337     static const auto inicio_coletor = std::chrono::steady_clock::now();
338
339     // Variáveis para cálculo online de confiança
340     // Mais medições numa mesma zona de posição → maior confiança
341     static float zona_x_anterior = -999.0f;
342     static int contagem_zona = 0;
343
344     if (e || shm->c_encerrar) {
345         if (arquivo_aberto) {
346             std::fflush(arquivo_coletor);
347             std::fclose(arquivo_coletor);
348             arquivo_aberto = false;
349             arquivo_coletor = nullptr;
350             log_message("COLETOR", "Arquivo de coleta fechado.");
351         }
352         return;
353     }
354
355     if (!arquivo_aberto) {
356         arquivo_coletor = std::fopen("dados_coletados.csv", "a+");
357         arquivo_aberto = true;
358         if (arquivo_coletor == nullptr) {
359             log_message("COLETOR", "Erro ao abrir arquivo de coleta!");
360         } else {
361             log_message("COLETOR", "Arquivo de coleta aberto para escrita.");
362             std::fseek(arquivo_coletor, 0, SEEK_END);
363             if (std::ftell(arquivo_coletor) == 0) {
364                 std::fprintf(arquivo_coletor,
365                     "timestamp_ms;posicao_x;teto_filtrado_y;lidar_raw;"
366
367                     ↪ "encoder;aceleracao;velocidade;e_automatico;e_inspecao;confianca\n");
368                 std::fflush(arquivo_coletor);
369             }
370         }
371     }
372
373     std::vector<float> dados_coletados;
374     {
375         std::lock_guard<std::mutex> lock(sinc.mtx_nivel);
376         while (sinc.disp_nivel > 0) {
377             dados_coletados.push_back(sinc.BUFFER_NIVEL[sinc.LER_IDX_NIVEL]);
378             sinc.LER_IDX_NIVEL = (sinc.LER_IDX_NIVEL + 1) % ELEMENTOS_BUFFERS;
379             sinc.disp_nivel--;
380         }
381     }
382
383     if (!dados_coletados.empty()) {
384         executado[3]++;
385
386         // Cálculo online de confiança: mais medições na mesma zona → maior
387         ↪ confiança

```

```

386     if (std::abs(shm->posicao_x - zona_x_anterior) < 1.0f) {
387         contagem_zona = std::min(contagem_zona + 1, 10);
388     } else {
389         zona_x_anterior = shm->posicao_x;
390         contagem_zona = 1;
391     }
392     float confianca = contagem_zona / 10.0f;
393     shm->perfil_confianca = confianca; // disponibiliza para o publisher
    ↪ MQTT
394
395     // Média de todas as leituras acumuladas no buffer / evita duplicatas e
    ↪ não perde passos
396     float dado = 0.0f;
397     for (float v : dados_coletados) dado += v;
398     dado /= static_cast<float>(dados_coletados.size());
399     const auto agora = std::chrono::steady_clock::now();
400     const long long timestamp_ms =
401         std::chrono::duration_cast<std::chrono::milliseconds>(agora -
    ↪ inicio_coletor).count();
402
403     if (arquivo_coletor) {
404         const int velocidade_efetiva = shm->j_sp_velocidade +
    ↪ shm->o_aceleracao;
405         std::fprintf(arquivo_coletor,
    ↪ "%lld;%.2f;%.2f;%d;%d;%d;%d;%d;%d;%.2f\n",
406             timestamp_ms,
407             shm->posicao_x,
408             dado,
409             static_cast<int>(shm->i_lidar),
410             static_cast<int>(shm->i_encoder),
411             static_cast<int>(shm->o_aceleracao),
412             velocidade_efetiva,
413             static_cast<int>(shm->e_automatico),
414             static_cast<int>(shm->e_inspecao),
415             confianca);
416
417         if (++contador_coletor >= 10) {
418             contador_coletor = 0;
419             std::fflush(arquivo_coletor);
420         }
421     }
422 } else {
423     miss[3]++;
424 }
425
426 if ((executado[3] + miss[3]) % 100 == 0) {
427     std::cout << "[STATUS] Miss: ";
428     for (auto i : miss) std::cout << i << " ";
429     std::cout << "| Exec: ";
430     for (auto i : executado) std::cout << i << " ";
431     std::cout << "\n";

```

```

432     }
433
434     reagendar_tarefa(t, PERIODO_COLETOR, "COLETOR");
435     t->async_wait(boost::asio::bind_executor(*strand,
436         std::bind(coletor_dados, std::placeholders::_1, t, strand, shm,
437             ↪ std::ref(sinc))));
438 }
439
440 void inspecao_camera(const boost::system::error_code& e,
441     ↪ boost::asio::steady_timer* t,
442         boost::asio::io_context::strand* strand, std::mutex& mtx,
443         ↪ MemoriaCompartilhada* shm) {
444
445     if (e) return;
446     if (shm->c_encerrar) return;
447
448     {
449         std::lock_guard<std::mutex> lock(mtx);
450         if (eventos_camera > 0) {
451             eventos_camera--;
452
453             log_message("CAMERA", "Inspeção iniciada | processamento de imagem
454                 ↪ em curso");
455
456             // Simula carga de CPU real: análise de imagem por visão
457             ↪ computacional embarcada
458             volatile float acumulador = 0.0f;
459             for (int i = 1; i <= 800000; ++i) {
460                 acumulador += std::sqrt(static_cast<float>(i)) *
461                     std::sin(static_cast<float>(i) * 0.0001f);
462             }
463             (void)acumulador; // impede otimização do compilador
464
465             log_message("CAMERA", "Inspeção finalizada");
466         }
467     }
468
469     reagendar_tarefa(t, PERIODO_CAMERA, "CAMERA");
470     t->async_wait(boost::asio::bind_executor(*strand,
471         std::bind(inspecao_camera, std::placeholders::_1, t, strand,
472             ↪ std::ref(mtx), shm))));
473 }

```

5.5 sincronizacao.hpp

```

1  #ifndef SINCRONIZACAO_H
2  #define SINCRONIZACAO_H
3
4  #include "includes.hpp"
5  #include "shared_memory.hpp"
6

```

```

7  #define ELEMENTOS_BUFFERS 10
8
9  #define PERIODO_COMANDO 500
10 #define PERIODO_CONTROLE 80
11 #define PERIODO_DISTANCIA 20
12 #define PERIODO_CAMERA 80
13 #define PERIODO_COLETOR 100
14 #define PERIODO_RECONSTRUCAO 80
15
16 struct VarCondSinc {
17     // BUFFER 1: NAVEGAÇÃO
18     std::vector<float> BUFFER_NAVEGACAO =
19         ↪ std::vector<float>(ELEMENTOS_BUFFERS);
20     int ESC_IDX_NAVEG = 0;    // Onde o produtor escreve
21     int LER_IDX_NAVEG_C = 0;  // Onde o Controle lê
22     int LER_IDX_NAVEG_R = 0;  // Onde a Reconstrução lê
23
24     int disp_naveg_c = 0;     // Quantidade de dados pendentes para o
25         ↪ Controle
26     int disp_naveg_r = 0;     // Quantidade de dados pendentes para a
27         ↪ Reconstrução
28     std::mutex mtx_navegacao; // Proteção exclusiva deste buffer
29
30     // === BUFFER 2: NÍVEL ===
31     std::vector<float> BUFFER_NIVEL = std::vector<float>(ELEMENTOS_BUFFERS);
32     int ESC_IDX_NIVEL = 0;    // Onde a Reconstrução escreve
33     int LER_IDX_NIVEL = 0;    // Onde o Coletor lê
34
35     int disp_nivel = 0;       // Quantidade de dados pendentes para o Coletor
36     std::mutex mtx_nivel;    // Proteção exclusiva deste buffer
37
38     // === CÂMERA ===
39     std::mutex mutex_camera;
40 };
41
42 typedef boost::asio::steady_timer tempo_tarefa;
43 typedef boost::asio::chrono::milliseconds milisegundos;
44
45 void log_message (const std::string& thread, const std::string& mensagem);
46 float numero_aleatorio_debugg();
47
48 void handler_signal (const boost::system::error_code& error, int signal_number,
49     ↪
50     boost::asio::steady_timer* t,
51     ↪ boost::asio::io_context::strand* strand_camera,
52     std::mutex& mtx, MemoriaCompartilhada* shm,
53     ↪ boost::asio::signal_set* sinais);
54
55 /**
56  * @brief Tarefa que recebe comandos do sistema de operação remoto e traduz os
57  * ↪ comandos em setpoint de velocidade

```

```

51  * e liga/desliga para o controle de navegação. O comando de navegação que
    ↳ deve implementar a lógica de manual/
52  * automático. Exerce a função de ESCRITOR sobre o BUFFER_NAVEGACAO
53  *
54  */
55  void comando_navegacao(const boost::system::error_code& /*e*/,
56                        boost::asio::steady_timer* t,
57                        boost::asio::io_context::strand* strand,
58                        MemoriaCompartilhada* shm);
59
60  /**
61   * @brief implementação de um controlador PID responsável pelo acionamento dos
    ↳ motores (controle de velocidade)
62   * a partir de um setpoint de velocidade recebido pelo Comando de Navegação.
    ↳ Exerce a função de LEITOR do BUFFER _NAVEGACAO
63   *
64   * @param mtx mutex utilizado
65   * @param BUFFER historico de posições
66   */
67  void controle_navegacao(const boost::system::error_code& e,
68                          boost::asio::steady_timer* t,
69                          boost::asio::io_context::strand* strand,
70                          MemoriaCompartilhada* shm, VarCondSinc &sinc);
71  /**
72   * @brief Registra a distância percorrida fornecida pelo encoder. Possui a
    ↳ função de ESCRITOR sobre o BUFFER_NAVEGACAO.
73   *
74   * @param mtx mutex utilizado na variavel compartilhada
75   * @param BUFFER historico de posições
76   */
77  void distancia_percorrida(const boost::system::error_code& e,
78                           boost::asio::steady_timer* t,
79                           boost::asio::io_context::strand* strand,
80                           MemoriaCompartilhada* shm, VarCondSinc &sinc);
81
82  /**
83   * @brief Recebe os valores do lidar para reconstruir o teto do túnel. Atua
    ↳ como
84   * ESCRITOR do BUFFER_NIVEL
85   *
86   * @param mtx mutex para buffer compartilhado
87   * @param BUFFER vetor de dados do nível da distancia do teto
88   */
89  void reconstrucao_teto(const boost::system::error_code& e,
90                        boost::asio::steady_timer* t,
91                        boost::asio::io_context::strand* strand,
92                        MemoriaCompartilhada* shm, VarCondSinc &sinc);
93
94  /**
95   * @brief Registra os dados coletados pelo lidar num Banco de Dados. Atua como
    ↳ LEITOR do BUFFER_NIVEL.

```

```

96  *
97  * @param mtx mutex para sincronizar o buffer compartilhado
98  * @param BUFFER buffer de dados coletados pelo lidar
99  */
100 void coletor_dados(const boost::system::error_code& e,
101                   boost::asio::steady_timer* t,
102                   boost::asio::io_context::strand* strand,
103                   MemoriaCompartilhada* shm, VarCondSinc &sinc);
104
105 void inspecao_camera(const boost::system::error_code& e,
106                    boost::asio::steady_timer* t,
107                    boost::asio::io_context::strand* strand,
108                    std::mutex& mtx, MemoriaCompartilhada* shm);
109
110 void operacao_remota(std::mutex &mtx, std::vector <float> &BUFFER);
111
112 #endif

```

5.6 shared_memory.hpp

```

1  #ifndef SHARED_MEMORY_HPP
2  #define SHARED_MEMORY_HPP
3
4  #include "include/includes.hpp"
5  #include <cstdint>
6
7  constexpr const char* SHM_FILE = "/tmp/memoria_compartilhada.bin";
8
9  struct MemoriaCompartilhada {
10
11     //sensores e atuadores disponíveis no carrinho:
12     bool i_encoder; //Variável que simula a entrada de um encoder, que troca de
13     ↪ estado a cada metro percorrido pelo robô
14     int32_t i_lidar; //Resposta do sensor LIDAR do veículo exibindo a distância
15     ↪ no eixo y, com relação à altura do robô
16     bool o_liga_camera; //Comando para ligar a câmera e realizar fotos da falha
17     ↪ detectada na superfície
18     int32_t o_aceleracao; //Determina a aceleração do veículo em percentual
19     ↪ (-100 a 100%)
20
21     //estados e comandos:
22     bool e_inspecao; //Estado que indica falha detectada na superfície e o robô
23     ↪ está tirando fotos e navegando com velocidade limitada (1:falha, 0: sem
24     ↪ falha)
25     bool e_automatico; //Estado que identifica o modo de operação do robô (0:
26     ↪ manual, 1: automático)
27     bool c_automatico; //Comando para passar o robô para o modo automático
28     ↪ (true). O reset desse comando
29     bool c_man; //Comando para passar o robô para o modo manual (true).
30     int32_t j_sp_velocidade; //Setpoint de velocidade do robô para o
31     ↪ controlador de velocidade.

```

```

23
24     bool c_encerrar; //variável que sinaliza ao programa que a interface foi
        ↪ fechada
25
26     // Parâmetros configuráveis via MQTT
27     float variacao_severa; // limiar de variação do teto que dispara inspeção
        ↪ (configurável remotamente)
28
29     // Dados de perfil do teto para publicação MQTT
30     float posicao_x; // posição acumulada do robô em metros (atualizada
        ↪ pelo encoder)
31     float perfil_y; // última leitura filtrada do teto (média móvel)
32     float perfil_confianca; // nível de confiança da medição (0.0 a 1.0)
33     bool perfil_novo; // sinaliza ao publisher que há novo ponto de
        ↪ perfil disponível
34 };
35
36 #endif

```

5.7 operacao_remota.cpp

```

1  #include "../include/includes.hpp"
2
3  void operacao_remota(std::mutex& mtx, std::vector<float>& BUFFER){
4      std::unique_lock<std::mutex> lock(mtx); // Protocolo de entrada
5      lock.unlock(); // Protocolo de saída
6  }

```

5.8 Makefile

```

1  TARGET = programa
2
3  CXX = g++
4
5  CXXFLAGS = -std=c++17 -Wall -Wextra -I. -Iinclude -I/opt/homebrew/include
        ↪ -I/usr/local/include -I/opt/homebrew/opt/boost/include -pthread
6
7  LDFLAGS = -L/opt/homebrew/lib -L/usr/local/lib \
8  -lpaho-mqttpp3 -lpaho-mqtt3c -lpthread \
9  -Wl,-rpath,/opt/homebrew/lib \
10 -Wl,-rpath,/usr/local/lib
11
12  SRCS = main.cpp src/sincronizacao.cpp src/simulacao.cpp src/mqtt_client.cpp
        ↪ src/mqtt_publisher.cpp src/mqtt_subscriber.cpp src/mqtt_manager.cpp
13
14  OBJS = $(SRCS:.cpp=.o)
15
16  all: $(TARGET)
17
18  $(TARGET): $(OBJS)
19      $(CXX) $(CXXFLAGS) $(OBJS) $(LDFLAGS) -o $(TARGET)

```



```
20
21 %.o: %.cpp
22     $(CXX) $(CXXFLAGS) -c $< -o $@
23
24 run: $(TARGET)
25     ./$$(TARGET)
26
27 clean:
28     rm -f $(OBJS) $(TARGET)
29
30 rebuild: clean all
```